

PORTADA

Arquitectura de Software

Semana 7 — Catálogo POSA: Layers, Microkernel

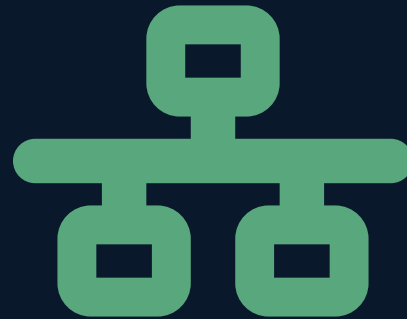
Universidad de Antioquia · Ingeniería de Sistemas · 2554608 · 2026-2



APERTURA

7 capas.

Más de 40 años sin romperse.



El modelo OSI, ISO 1984

Nombre	Open Systems Interconnection (OSI)
Organismo	International Organization for Standardization (ISO)
Año de estandarización	1984
Objetivo	Estandarizar cómo se comunican sistemas de cómputo distintos, dividiendo el problema en niveles independientes
Estructura	7 capas apiladas, cada una con una responsabilidad propia
Vigencia	Sigue siendo la referencia conceptual estándar para enseñar y diseñar redes, más de cuarenta años después

Estándar documentado por ISO/IEC 7498; es, probablemente, el ejemplo más citado de arquitectura en capas de toda la industria de sistemas.

CASO · LAS 7 CAPAS

De los bits al programa que usa la red

#	Capa	Responsabilidad
1	Física	Transmitir bits sobre un medio físico (cobre, fibra, radio)
2	Enlace de datos	Entregar tramas de datos sin errores entre nodos directamente conectados
3	Red	Enrutar paquetes entre redes distintas (ej. IP)
4	Transporte	Garantizar entrega confiable extremo a extremo (ej. TCP)
5	Sesión	Abrir, mantener y cerrar sesiones de comunicación
6	Presentación	Traducir formatos de datos (cifrado, compresión, codificación)
7	Aplicación	Protocolos que usan directamente los programas (HTTP, SMTP, DNS)

CASO · POR QUÉ FUNCIONÓ EL DISEÑO EN CAPAS

Cada capa evoluciona sin tocar a las demás

La razón por la que el modelo OSI se sostiene décadas después no es que las siete capas sean "las correctas" — es que **cada capa solo depende de la capa inmediatamente adyacente**, nunca de los detalles internos de las demás.

- La capa física pasó de cable de cobre a fibra óptica y a redes inalámbricas **sin que las capas superiores tuvieran que cambiar**.
- La capa de aplicación ganó protocolos nuevos (HTTP/2, HTTP/3) sin rediseñar el transporte o la red debajo.
- Fabricantes distintos pueden construir hardware y software para una sola capa, confiando en que las interfaces con las capas vecinas se mantienen estables.

Aislar el cambio en cada nivel es lo que permitió que protocolos y tecnologías completamente distintos evolucionaran en paralelo, durante décadas, sin romper el sistema completo.

TRANSICIÓN

El mismo principio, aplicado al software empresarial

OSI organiza *redes* en capas. Pero el mismo principio de aislamiento por niveles —cada capa solo conoce a su vecina inmediata — se aplica también a la **arquitectura interna de una aplicación**: capas de presentación, lógica de negocio y persistencia, como ya vieron con MVC y con arquitecturas cliente/servidor N-capas en semanas anteriores.

Hoy formalizamos ese principio general con nombre propio: el patrón **Layers** del catálogo POSA.

CONCEPTO

Layers: organizar el sistema en niveles

El patrón **Layers** (Buschmann et al., *Pattern-Oriented Software Architecture*) organiza el sistema en **niveles jerárquicos**. Cada capa:

- Ofrece un conjunto de **servicios** a la capa inmediatamente superior.
- Consume los servicios que le ofrece la capa inmediatamente inferior.
- No conoce ni depende de capas que no sean sus vecinas directas.

Es exactamente lo que ilustra OSI: la capa de transporte usa servicios de red, y ofrece servicios a la capa de sesión — nunca "salta" capas.

CONCEPTO

El principio de aislamiento de cambios

La razón de ser del patrón es el **principio de aislamiento de cambios**: si una capa cambia su implementación interna sin cambiar la interfaz que expone, ninguna otra capa debería enterarse.

En una aplicación empresarial típica esto se traduce en tres capas:

- **Presentación** — interfaz de usuario o API expuesta al exterior.
- **Lógica de negocio** — reglas y procesos propios del dominio.
- **Persistencia** — acceso a la base de datos u otro almacenamiento.

Cambiar el motor de base de datos no debería obligar a tocar la capa de presentación — el mismo principio que permitió pasar de cobre a fibra en OSI, aplicado dentro de una sola aplicación.

CONCEPTO · VENTAJAS Y LIMITACIONES

Lo que Layers gana — y lo que cuesta

Ventajas	Limitaciones
Bajo acoplamiento entre capas: cada una se puede reemplazar o probar de forma aislada	Cada capa añade una llamada adicional: puede introducir overhead de rendimiento frente a acceder directamente al nivel inferior
Facilita el mantenimiento: un cambio queda contenido en su capa	Las capas a veces se "filtran" (<i>leaky abstractions</i>): un detalle de la capa inferior termina siendo visible o necesario en la superior, rompiendo el aislamiento prometido
Facilita las pruebas: se puede simular (mockear) la capa vecina	Diseñar demasiadas capas "por si acaso" agrega complejidad sin beneficio real

Leaky abstraction ("abstracción con fugas"): cuando una capa que debía ocultar sus detalles internos obliga, en la práctica, a que la capa superior conozca esos detalles para funcionar bien.

TRANSICIÓN

¿Y si el sistema no se puede predecir por completo desde el diseño?

Layers funciona bien cuando las responsabilidades del sistema se conocen de antemano y se pueden ordenar en niveles. Pero hay sistemas que necesitan crecer con funcionalidad **que nadie previó al diseñarlos** — nuevos lenguajes, nuevas herramientas, nuevos formatos — sin reescribir el sistema cada vez.

Para ese problema, el catálogo POSA propone un patrón distinto: Microkernel.

Eclipse IDE

Producto	Eclipse — entorno de desarrollo integrado (IDE)
Origen	Iniciado por IBM, lanzado como código abierto en 2001
Gobernanza actual	Eclipse Foundation, organización sin ánimo de lucro creada en 2004 para gobernar el proyecto
Sistema de módulos	Construido sobre OSGi , un estándar de módulos dinámicos para Java
Núcleo	Mínimo: apenas lo necesario para cargar y coordinar plugins

OSGi: especificación que permite empaquetar código Java en módulos ("bundles") que se pueden instalar, iniciar, detener y actualizar en caliente, sin reiniciar la aplicación completa.

CASO · CASI TODO ES UN PLUGIN

El núcleo hace poco, a propósito

El núcleo de Eclipse resuelve un problema muy acotado: cargar módulos y dejar que se comuniquen entre sí a través de **puntos de extensión** bien definidos. Todo lo demás —lo que un usuario reconoce como "Eclipse"— es un **plugin**:

- Soporte para cada lenguaje de programación (Java, C++, PHP, ...)
- Herramientas de depuración (*debugging*)
- Integración con control de versiones (Git, SVN)
- Temas visuales y personalización de la interfaz

Plugin (extensión): módulo independiente que agrega funcionalidad a un sistema, comunicándose con el núcleo a través de una interfaz definida por el propio núcleo, sin modificar su código.

CASO · POR QUÉ FUNCIONÓ ESTE DISEÑO

Dos décadas sin reescribir el núcleo

Como el núcleo de Eclipse nunca necesitó conocer de antemano qué lenguajes o herramientas soportaría, el proyecto pudo **sumar capacidades completamente nuevas** —lenguajes que no existían en 2001, nuevas prácticas de control de versiones— sin tocar la base del sistema.

El mismo patrón explica por qué otros sistemas longevos y muy extensibles se diseñan igual: Visual Studio Code con su sistema de extensiones, o los navegadores web con sus extensiones instalables, aplican la misma idea de núcleo mínimo más plugins.

Un núcleo pequeño, bien definido, es más fácil de mantener estable durante veinte años que uno que intenta prever todo desde el diseño.

CONCEPTO

Microkernel: mecanismo, no política

El patrón **Microkernel** separa el sistema en dos partes:

- Un **núcleo mínimo** con solo la funcionalidad esencial — el "mecanismo" que hace posible que el sistema funcione.
- Toda la funcionalidad adicional, implementada como **plugins** que se conectan al núcleo mediante una interfaz bien definida.

"Mechanism, not policy": el núcleo provee el mecanismo genérico para cargar y coordinar extensiones, pero no decide *qué* hace cada extensión — esa decisión ("la política") vive en los plugins, fuera del núcleo.

CONCEPTO

Cómo se comunican núcleo y plugins

El núcleo expone **puntos de extensión** (interfaces conocidas de antemano); cada plugin implementa esos puntos para agregar su funcionalidad. El núcleo no necesita saber, al diseñarse, cuáles plugins existirán en el futuro — solo necesita mantener estable la interfaz.

- Se puede **agregar o quitar un plugin sin modificar el núcleo** ni los demás plugins.
- Un plugin que falla puede aislarse, sin necesariamente derribar el núcleo ni el resto del sistema.



CONCEPTO · VENTAJAS Y LIMITACIONES

Lo que Microkernel gana — y lo que cuesta

Ventajas	Limitaciones
Extensibilidad sin modificar el núcleo: se agregan capacidades nuevas como plugins	Coordinar el comportamiento entre varios plugins independientes puede volverse complejo
Aislamiento de fallos: un plugin defectuoso no necesariamente compromete el núcleo	La comunicación núcleo–plugin, si es muy genérica, puede afectar el rendimiento
Permite que terceros extiendan el sistema sin acceso a su código fuente	El diseño del núcleo y sus puntos de extensión debe anticiparse bien — cambiar esa interfaz después es costoso

TENSIÓN CONCEPTUAL

Orden vertical estricto vs. relación núcleo-radial

Layers impone un orden de dependencia estrictamente vertical: cada capa solo puede hablar con la capa inmediatamente adyacente, arriba o abajo. Saltarse ese orden rompe el patrón.

Microkernel no tiene niveles ordenados entre sí: es una relación **núcleo-radial** — todos los plugins hablan con el núcleo, pero no necesariamente entre ellos ni en un orden predefinido.

Ambos persiguen lo mismo —bajo acoplamiento y capacidad de evolucionar sin romper el sistema— pero con formas de dependencia opuestas: una jerárquica y lineal, la otra centrada en un núcleo común.



INTERACCIÓN

¿Layers o Microkernel?

Para cada sistema, decidan si se organiza mejor como Layers, como Microkernel, o con ambos combinados, y justifiquen:

1. Un sistema bancario con capas claras de presentación, reglas de negocio y persistencia, sin necesidad de extensiones de terceros.
2. Un navegador web que debe permitir instalar extensiones creadas por cualquier desarrollador externo.
3. Un sistema operativo que separa un núcleo mínimo de gestión de recursos de todo lo demás (drivers, servicios).

5 minutos · respondan en voz alta o en el chat

SÍNTESIS

Ideas clave de hoy

1. **Layers** organiza el sistema en niveles jerárquicos donde cada capa solo conoce a su vecina inmediata — el modelo OSI (ISO, 1984) es el caso canónico de la industria.
2. El aislamiento por capas permite que tecnologías distintas evolucionen en paralelo sin romper el sistema completo, aunque introduce overhead y riesgo de *leaky abstractions*.
3. **Microkernel** separa un núcleo mínimo ("mecanismo, no política") de funcionalidad adicional implementada como plugins — Eclipse IDE, sobre OSGi, es el caso de referencia.
4. Microkernel da extensibilidad y aislamiento de fallos, a cambio de mayor complejidad de coordinación entre plugins.
5. Layers depende de un orden vertical estricto; Microkernel es núcleo-radial — dos caminos distintos hacia el mismo objetivo: bajo acoplamiento y capacidad de evolucionar.

Con Layers y Microkernel completamos ocho patrones del catálogo POSA vistos en la Unidad 2: Pipes & Filters, Blackboard, Broker, MVC, Batch, Cliente/Servidor N-capas, Layers y Microkernel.

CIERRE

Para la próxima semana

- Repasar los ocho patrones POSA vistos hasta ahora en la Unidad 2 y en cuáles proyectos reales (propios o conocidos) reconocen cada uno.
- Identificar, en el proyecto de Fábrica Escuela, si hoy existe algo parecido a capas o a un núcleo extensible con plugins.

La Semana 8 continúa el catálogo POSA con dos patrones más: Arquitectura Orientada a Servicios (SOA) y Arquitectura Hexagonal.

